

Logic Bubble — Project Brief (Strategy & Scope)

1. Executive Summary

Logic Bubble is a visual logic-mapping web application designed for readers, students, and knowledge workers who need to extract and organise conceptual structure from dense text. Users paste any body of text into the editor, pull out key phrases as interactive circular "bubble" nodes on a pan-and-zoom canvas, and connect those nodes with directed arrows to represent relationships, sequences, or dependencies. The application targets anyone who regularly works with complex narrative or conceptual frameworks — technical documentation, academic papers, legal analysis, or long-form fiction — and needs a lightweight, browser-native tool to build and persist those mental models.

2. Problem Statement

Reading long, complex texts — technical documentation, academic PDFs, novels, research reports — makes it genuinely difficult to keep track of how concepts, characters, or arguments relate to one another across hundreds of pages. Readers must either hold an increasingly fragile mental model or resort to external tools that add friction to the workflow.

Existing solutions fall into two categories, neither of which is adequate:

- Heavyweight diagramming and collaboration tools (Miro, Figma, Notion, MindMeister) provide rich functionality but require significant setup, are priced for teams, and are not purpose-built for the reading-and-extraction workflow.
- Physical and analogue methods (sticky notes, whiteboard sketches, index cards) are fast and tactile but are not persistent, not searchable, and not shareable in a structured format.

There is currently no lightweight, browser-based tool built specifically for constructing visual logic maps directly from pasted source text, with the source text and the diagram stored together as a single named project. Logic Bubble fills that gap.

3. Target Audience

The application is designed for the following primary user groups:

- University students reading academic literature, tracking arguments across multiple papers, or building study diagrams for complex theoretical frameworks.
- Researchers and analysts working with technical documentation, legal briefs, or policy papers who need to surface causal chains and dependency structures.
- Fiction readers tracking complex plots, character relationships, and narrative timelines across long-form novels or serialised media.
- Knowledge workers — consultants, product managers, systems analysts — who build mental models from reports, meeting notes, and strategic documents and need to externalise and persist those

models.

All target users share a common profile: they are comfortable in a browser, they work with text-heavy material, and they need a fast on-ramp with minimal setup or account overhead.

4. Market Analysis

Competitive Landscape

Tool	Strengths	Weaknesses Relative to Logic Bubble
Miro	Rich canvas, real-time collaboration, templates	Cloud-dependent, team-oriented pricing, no text-extraction workflow, heavyweight onboarding
Notion	Flexible documents, database views, integrations	Not a diagramming tool; no canvas; requires structured data entry
MindMeister	Purpose-built mind mapping, good UX	Hierarchical tree structure only; no free-form canvas; no source-text binding
draw.io / diagrams.net	Powerful, free, exportable	Generic diagramming; not optimised for text extraction; no backend persistence
Obsidian (Graph View)	Local-first, privacy-respecting, linked notes	Note-based, not visual canvas; no on-the-fly extraction from pasted text

Logic Bubble is purpose-built for a single, focused workflow: paste text, extract concepts visually, save. It is not trying to compete on breadth of features; it competes on friction-reduction for this specific task.

Future Opportunity

The architecture of Logic Bubble positions it well for two high-value future integrations:

- Embedding into existing reading environments — Kindle-style e-reader apps, browser-based PDF readers, or learning management systems (LMS) could embed Logic Bubble as a native annotation layer, allowing users to build diagrams without leaving the reading context.
 - AI-assisted diagram generation — With a language model backend, uploaded documents could be automatically analysed to produce a first-draft bubble diagram, which the user then refines manually. This would dramatically reduce time-to-diagram for long documents.
-

5. Product Goals (v4.1.x Scope)

The goals for the current version are deliberately scoped to core utility and stability:

- Provide a smooth, keyboard-driven canvas editor for creating, editing, moving, connecting, and deleting bubble nodes and connection lines — with full mouse and keyboard parity for all operations.

- Persist projects per authenticated user via a secure Flask/MySQL backend, with named projects, explicit save (Ctrl+S), and full reload fidelity (no data loss on roundtrip).
 - Support colour, font, and layout customisation including bubble colour and opacity, font family and size, connection line style, and focus ring colour — all persisted in user preferences.
 - Maintain a clean, accessible, responsive UI for desktop browsers, adhering to semantic HTML, visible focus states, and ARIA best practices.
 - Provide a complete user system including registration, login, session management, dashboard for project management, and an admin panel for contact messages.
-

6. Scope

6.1 In Scope (Current Version)

Canvas & Navigation

- Pan the canvas with Space+drag or middle-mouse drag
- Zoom with the scroll wheel (centred on cursor)
- Reset view to fit content with the H key

Nodes

- Bubble nodes: circular nodes containing editable text; radius auto-sized to content
- Dot nodes: small circular junction/waypoint nodes with no text
- Create a new node with Enter (when no node is focused) or double-click on blank canvas
- Edit node text inline with F2 or double-click on the node
- Delete selected node with Del
- Merge nodes by dragging one bubble over another (content concatenated)
- Snap alignment guides displayed when dragging nodes near horizontal or vertical alignment

Connections

- SVG arrow lines connecting any two nodes
- Toggle between straight lines and cubic Bézier curves (Shift+S or preferences toggle)
- Drag either endpoint of a connection to retarget it to a different node
- Drag a connection's midpoint to blank canvas space to delete the connection

Customisation

- Per-bubble colour picker (full HSL spectrum) and opacity slider
- Quick colour preset swatches for common colours
- Focus ring colour selector — separate colours for select mode and connect mode
- Font family selector (system fonts)
- Font size controls for bubble text and editor UI text
- Line style preference (straight vs. curved) persisted to user preferences

Keyboard System

- Comprehensive keyboard shortcut set covering all major canvas operations

- All shortcuts customisable via the Keyboard Shortcuts modal (persisted to localStorage)
- Shortcut hints overlay (toggle with I) displayed bottom-left of canvas

User System

- User registration and login with hashed passwords (Werkzeug)
- Session-based authentication with role support (user / admin)
- Dashboard: create new project, view all user projects, edit, export, delete
- Ctrl+S to save the current project from the editor
- Ctrl+Z undo for the most recent canvas operation

Pages

- Home: public landing page with hero card and primary CTA
- Dashboard: authenticated project management grid
- Editor: full-screen canvas editor (standalone page, no shared base layout)
- Documentation: public reference page for all features and shortcuts
- About: project description and team information
- Contact form: submits messages stored in the database
- Admin Messages: panel listing all contact form submissions (admin role only)

UX Details

- Right-click context menu on nodes and canvas (mode-aware)
- Toast notification system for save confirmations, errors, and status messages
- Help modal with feature overview
- Dark theme throughout (single theme for v4.1.x)

6.2 Out of Scope (Future Roadmap)

The following features are explicitly deferred to future versions:

Feature	Notes
AI-powered diagram generation	Requires LLM API integration; planned for a future milestone
PDF import and in-page annotation	Significant additional complexity; separate sub-project scope
Image upload and embed	Not a core logic-mapping feature; future enhancement
Light theme	UI token system supports it structurally; deferred by design decision
Mobile and touch support	Canvas interaction model is fundamentally mouse-dependent
Real-time collaboration	Requires WebSocket infrastructure; out of scope for single-user MVP
Version history / branching	Undo history is session-only; persistent versioning is a future feature

Feature	Notes
Export to image / SVG	Planned; not implemented in current version

7. Technology Stack

Layer	Technology	Notes
Backend	Python 3, Flask 3.1	Application server and routing
Database driver	mysql-connector-python	Direct SQL queries; no ORM
Password security	Werkzeug security utilities	generatepasswordhash / checkpasswordhash
Database	MySQL / MariaDB	Database name: logicbubbleapp
Frontend logic	Vanilla JavaScript (ES Modules)	No framework; modular file structure
Templating	Jinja2 (via Flask)	Server-side rendering for all pages
Styling	Custom CSS (style.css, theme.css, base.css) + Tailwind CSS	Custom properties for design tokens; Tailwind for utility classes
UI fonts	Syne	Site navigation, headings, and body text
Editor font	JetBrains Mono	Topbar and editor-specific UI elements
Font delivery	Google Fonts (CDN)	Loaded via <link> in templates

The decision to use Vanilla JS with ES Modules rather than a frontend framework was deliberate: it avoids build tooling complexity, keeps the dependency surface minimal, and matches the scope of a focused single-page canvas tool.

8. Success Metrics

The following criteria define a successful delivery of v4.1.x:

Criterion	Definition of Done
Data integrity	A user can create a project with bubbles and connections, save it (Ctrl+S), navigate away, return, and reload it with all nodes, positions, colours, and connections intact — no data loss.
Keyboard accessibility	All core canvas operations (create, edit, move, connect, delete, save, undo) are fully accessible via keyboard without requiring a mouse.
Performance	Initial page load on localhost is under 2 seconds for all pages, including the editor, on a modern desktop browser.

Criterion	Definition of Done
Security	All user-supplied content rendered in the browser is escaped via Jinja2 auto-escaping; no XSS vectors are present. Passwords are never stored in plaintext.
Robustness	The application handles empty states (no projects, empty canvas), invalid inputs, and session expiry gracefully with informative user feedback.